MAKE MY MARRIAGE
Free-Tier System Design

A cost-conscious architecture for a portfolio project with a path to a future launch

React + TypeScript + Zod • Java Spring Boot • PostgreSQL • Vercel • Free-tier services

Revision 2.0 | 20 October 2026

Goal: avoid paid infrastructure. Free plans have limits and can change; this design avoids relying on AWS services that may start billing after credits or trial allowances end.

1. What changed and why

The earlier design used AWS ECS/Fargate, RDS PostgreSQL, S3 and SES. These are pay-as-you-go services. AWS may offer temporary credits, but credits are not a permanent free plan. See AWS Free Tier details and RDS PostgreSQL pricing.

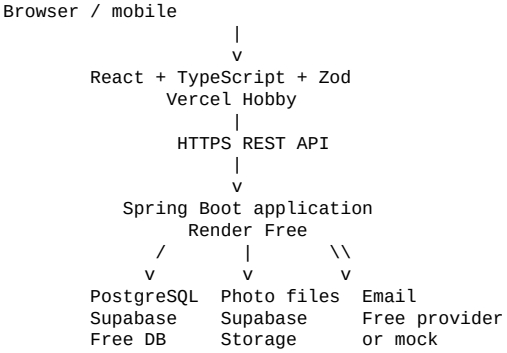
Recommended free-tier-first stack

Layer	Use for V1	Caveat
Frontend	Vercel Hobby	Free for personal/non-commercial use only; not for a commercial launch. Official plan details.
Backend	Render free web service for Spring Boot	Can sleep when idle; first request may be slow. Check current quotas: Render free plan.
Database	Supabase Free PostgreSQL	Free quotas, inactivity and project policies apply.
Photo storage	Supabase Storage Free initially	Replaces S3 to reduce risk of AWS charges; check current storage/egress limits.
Email	Free email allowance or mocked email in development	Verify current provider quota and sender/domain rules. Do not assume SES is permanently free.
Validation / auth	Zod + backend Bean Validation; Spring Security + JWT	Validate again on backend; keep tokens short-lived.

Why not AWS for every component?

A zero-cost-first project should not depend on services that can start charging when credits expire or usage exceeds allowances. Use AWS later as a deliberate learning/deployment phase if you want to practice AWS.

2. Architecture diagram



Responsibilities

Component	Responsibility
React	Screens, forms, navigation, responsive UI and client-side Zod schemas.
Spring Boot	REST API, business rules, membership/role checks, database access, photo authorization and email adapter.
PostgreSQL	Users, weddings, roles, events, guests, RSVPs, expenses, vendors and photo metadata.
Object storage	Photo files only; keep private and store object keys/status in PostgreSQL.
Email	Sends invitations and notifications through a replaceable EmailService interface.

If the free storage product does not support your preferred upload flow, start with organizer-only uploads and add QR guest uploads after testing. Do not make a bucket public for convenience.

3. Request and photo-upload flows

Normal request

```
1. React calls GET /api/v1/events over HTTPS.
2. Spring Security validates the JWT.
3. Backend checks wedding membership and role.
4. Service reads authorized rows from PostgreSQL.
5. API returns JSON; React renders events.
```

Photo upload with moderation

```
1. Guest scans QR and opens the upload page.
2. Frontend asks Spring Boot for upload authorization.
3. Backend validates the link, event and upload limits.
4. Browser uploads to private object storage using the
   provider's supported signed upload mechanism.
5. Backend stores metadata with status PENDING.
6. Organizer approves or rejects the photo.
7. Only approved photos are shown to authorized viewers.
```

- Keep AWS/database secrets out of React variables.
- Validate file size, type and authorization on the backend; randomize object names.
- Use expiring/revocable QR links and rate limits.
- Enforce roles in Spring Boot, not only in React.
- Keep photos private and check approval before serving.

4. Repository structure

```
make-my-marriage/
  frontend/src/
    app/ components/
    features/
      auth/ dashboard/ events/ guests/
      expenses/ vendors/ photos/ livestream/ settings/
    lib/
      api-client.ts
      schemas/             # Zod schemas
    types/ styles/
  backend/src/main/java/com/makemymarriage/
    auth/ wedding/ events/ guests/
    expenses/ vendors/ photos/ email/
    livestream/ settings/ common/ config/
  backend/src/main/resources/
    db/migration/          # Flyway SQL
    application.yml
  backend/src/test/
  docs/
```

Keep the backend as a modular monolith. Use Controller → Service → Repository → PostgreSQL. Controllers use DTOs; services contain business rules; repositories access data. Use Flyway, Spring Security, Bean Validation and a global error handler.

Frontend defaults

• React + TypeScript + Vite; React Router for pages.

• Zod schemas for event, guest and expense forms; backend validation repeats checks.

• Use a shared API client and consistent loading/error states.

• VITE_API_BASE_URL is public configuration; never put secrets in VITE_ variables.

5. Database tables and REST APIs

Table	Purpose / relationship
users	Login identities; unique normalized email.
weddings	Wedding workspace and settings.
wedding_members	User-to-wedding role; unique wedding/user pair.
events / tasks	Events belong to a wedding; tasks belong to events.
guests / households	Wedding guest list and optional family grouping.
invitations / rsvps	Event-specific invitation token hash, status, expiry and response.
expenses	Category, amount, amount_paid, optional vendor; NUMERIC(12,2).
vendors / saved_vendors	Directory, search and wedding favourites.
photo_albums	Album associated with wedding/event.
photos	Object key, uploader, status PENDING/APPROVED/REJECTED and file metadata.
photo_upload_links	QR token hash, expiry and revocation.
email_jobs	Optional delivery/retry status.
audit_logs	Sensitive changes such as role changes and photo moderation.

API examples under /api/v1

Feature	Example routes
Auth	POST /auth/register, /auth/login, /auth/refresh, /auth/logout
Wedding	GET/PATCH /wedding; GET /wedding/members
Events/tasks	GET/POST /events; PATCH /events/{id}; GET/POST /events/{id}/tasks
Guests/invitations	GET/POST /guests; POST /invitations/email; POST /invitations/{token}/rsvp
Expenses	GET/POST /expenses; PATCH /expenses/{id}; GET /expenses/summary
Vendors	GET /vendors?city=&category;=; POST /vendors/{id}/favorites
Photos	POST /albums/{id}/upload-url; POST /photos/{id}/complete; POST /photos/{id}/approve or /reject
Livestream/theme	PATCH /events/{id}/livestream; PATCH /wedding/theme

6. Authentication and privacy

Role	V1 permissions
ADMIN	Bride/groom: manage workspace, members, events, guests, expenses, settings and moderation.
MANAGER	Family organizers: manage allowed planning features.
GUEST	RSVP and upload photos using valid invitation/QR link; no planning/expense access.

Security defaults

• Hash passwords with BCrypt or Argon2id.

• Access JWTs expire quickly (e.g. 10 minutes); validate signature, issuer, audience and expiry.

• Avoid long-lived JWTs in localStorage. If using refresh tokens, rotate/revoke them and store in Secure, HttpOnly, SameSite cookies; protect cookie-based changes against CSRF.

• Enforce membership and role checks in Spring Boot for every protected endpoint.

• Use HTTPS, rate limits, safe errors and least-privilege credentials.

• Keep photos private by default; define deletion and retention rules before real users.

7. Environment variables

Frontend Vercel variable

```
VITE_API_BASE_URL=https://your-render-service.example/api/v1
```

Backend variables

```
SPRING_PROFILES_ACTIVE=prod
DB_URL=<Supabase PostgreSQL JDBC URL>
DB_USERNAME=<from host secret settings>
DB_PASSWORD=<from host secret settings>
JWT_ISSUER=https://your-render-service.example
JWT_SIGNING_KEY=<strong-secret-from-host-secret-settings>
SUPABASE_URL=<project URL>
SUPABASE_SERVICE_ROLE_KEY=<backend-only secret>
EMAIL_PROVIDER=free-provider
EMAIL_API_KEY=<backend-only secret>
EMAIL_FROM=no-reply@your-verified-domain.example
FRONTEND_BASE_URL=https://your-project.vercel.app
CORS_ALLOWED_ORIGINS=https://your-project.vercel.app
```

Use placeholders in .env.example only. Keep Supabase service-role and email API keys on the backend. Never commit secrets.

8. Deployment steps

Frontend — Vercel

• Push frontend to GitHub and import into Vercel.

• Set VITE_API_BASE_URL to the deployed backend URL.

• Use Hobby only for personal/non-commercial portfolio work; its terms do not permit commercial use.

Database and photo storage — Supabase

• Create a Free project and apply schema migrations.

• Create private storage buckets; test upload/download permissions before QR uploads.

• Export important data periodically. Free demo hosting is not a substitute for tested backups.

Backend — Render

• Deploy Spring Boot from Git or a Dockerfile.

• Set backend secrets in Render's environment configuration.

• Allow CORS only from the Vercel origin and configure a health endpoint.

• Expect cold starts/sleep on free service; acceptable for a portfolio demo, not time-critical production.

Email

• Check the chosen provider's current free transactional-email allowance and sender verification requirements.

• During local development, mock/log email rather than sending real messages.

• Keep email behind an EmailService interface so you can switch providers later.

9. What 'completely free' means

This design aims for $0 recurring infrastructure cost within free-plan quotas, but no vendor can guarantee a free plan forever. Limits, eligibility and terms can change; payment details may be requested at signup, and accidental overuse can create risk.

Service	Caveat
Vercel Hobby	Free for personal/non-commercial use; not a commercial launch plan. Terms.
Render	Free services have limits and may sleep. Render Free PostgreSQL expires after 30 days, so do not use it as the durable database. Details.
Supabase	Free quotas, inactivity policies and storage/egress limits apply. Confirm current limits before relying on it. Pricing.
Email provider	Free sending quota and verification rules vary; confirm current plan before implementation.
AWS	Avoid AWS ECS/RDS/ALB/SES as default for this zero-cost build. AWS credits are temporary, not permanent free services. AWS Free Tier.

Cost safety checklist

• Do not upgrade to a paid plan during setup.

• Set usage alerts where available and review usage weekly.

• Limit photo upload size; storage and bandwidth can be the first quotas reached.

• Export data before a free project pauses or expires.

• Do not promise 24/7 availability from free services.

10. Phased implementation plan

Phase	Build	Acceptance check
1. Foundation	React + TS + Zod; Spring Boot; local PostgreSQL; Flyway.	Both apps run locally; migrations succeed.
2. Auth/workspace	Registration/login, JWT, workspace, membership and roles.	Unauthorized workspace access is blocked.
3. Events/themes	Events CRUD, checklist, dashboard summary, themes/settings.	Changes persist after refresh.
4. Guests/email	Guest CRUD, invitation tokens, RSVP, email adapter.	RSVP works; expired tokens fail safely.
5. Expenses/vendors	Expense CRUD/summary, vendor search/favourites.	Balances correct; workspace isolation tested.
6. Photos/livestream	Private storage, QR upload link, moderation, YouTube URL.	Pending photos hidden; only approved photos visible.
7. Free hosting	Vercel, Render, Supabase and email provider.	End-to-end demo works within free quotas.

Testing checklist

- Unit tests for expense totals, invitation tokens, role rules and photo moderation.
- Spring Boot integration tests for database access, migrations and endpoint security.
- React Testing Library for Zod forms, error states and role-based navigation.
- End-to-end: create workspace → event → invite → RSVP → upload → approve.
- Negative tests: guest reads expenses; wedding A accesses wedding B; expired QR; oversized image; unapproved photo.

Final recommendation

For the $0-first version, use Vercel for React, Render Free for Spring Boot, Supabase Free for PostgreSQL and photo storage, and a free email-provider allowance or mocked email during development. Add AWS later as a deliberate learning/deployment phase, knowing it may no longer be zero-cost.

Official references

AWS Free Tier: https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier.html
AWS RDS PostgreSQL pricing: https://aws.amazon.com/rds/postgresql/pricing/
Vercel Hobby: https://vercel.com/docs/plans/hobby
Vercel terms: https://vercel.com/legal/terms
Render free plan: https://render.com/docs/free
Supabase pricing: https://supabase.com/pricing